Description of the laboratory work No. 2

GENERAL INPUT/OUTPUT OF 32 BIT MICROCONTROLLERS. REGISTER LEVEL

1. Objectives
2. Preparation for the work
3. Workflow
   3.1. Work with a teacher
      3.1.1. STM32CubeIDE
      3.1.2. Template of the code
      3.1.3. Main functions
   3.2. An example of an individual laboratory work assignment
   3.3. Work by individual task
   3.4. Literature

1. Objectives

Get to know the architecture, functionality and configuration features of the general I/O of 32 bit microcontrollers. To develop abilities to analyze microcontrollers specifications. Get to know the software, which is needed for program of 32 bit microcontrollers in register level.

2. Preparation for the work

We will start to work with general input output of 32-bit microcontrollers in the register level during this laboratory work. Students will try to accomplish several tasks independently after getting to know the basics.

3. Workflow

3.1. Work with a teacher

The lecturer demonstrates boards, explains their components and students start to program individually.

3.1.1. STM32CubeMX

STM32CubeMX is a software package that allows you to perform project configuration work in the graphical interface. First, when you run the STM32CubeMX software for the first time, it will be necessary to create a new project

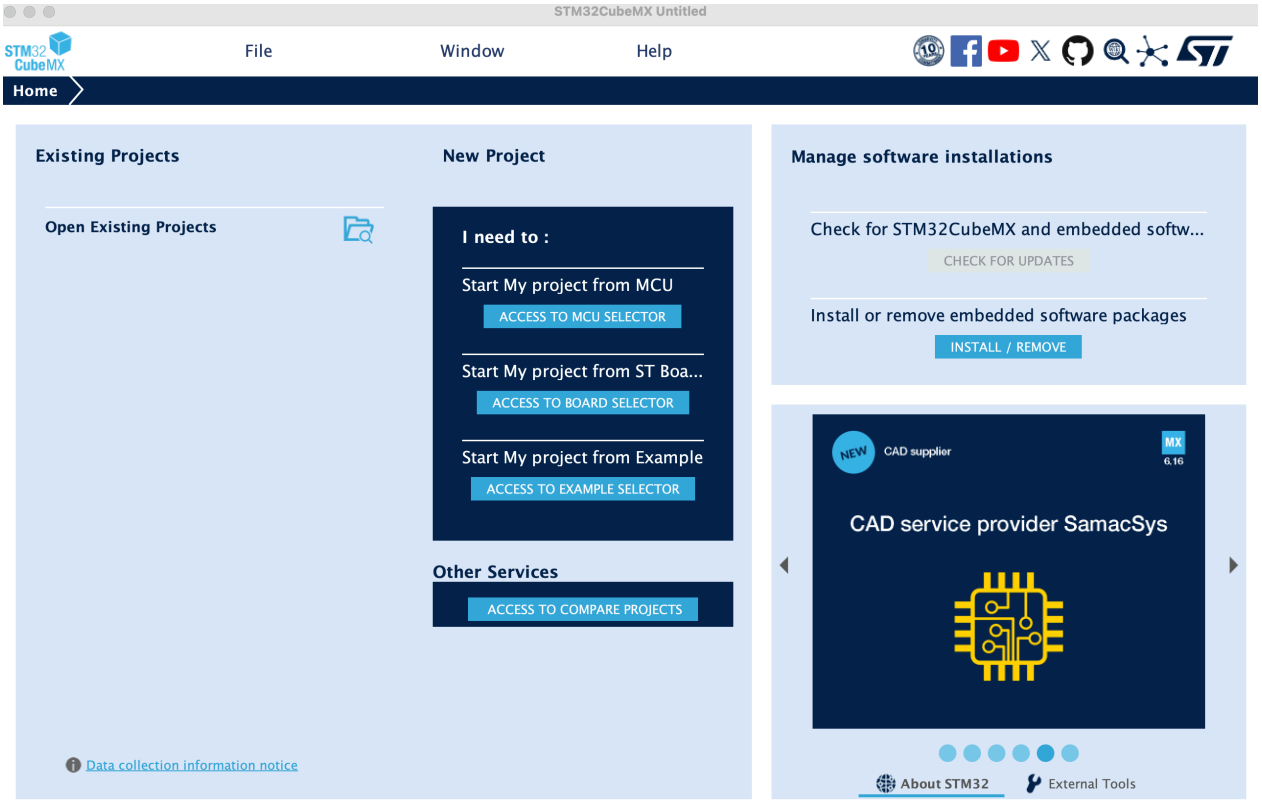


Fig. 1. The dialog window for selection of the home page

NOTE: The STM32CubeMX version may vary by laboratory, so the program window may look slightly different. Just have a look at where to choose the right configuration.

In the main configuration window, it is necessary to press the “Access to board selector” button. As a result, the board selection window will open (Fig. 2).

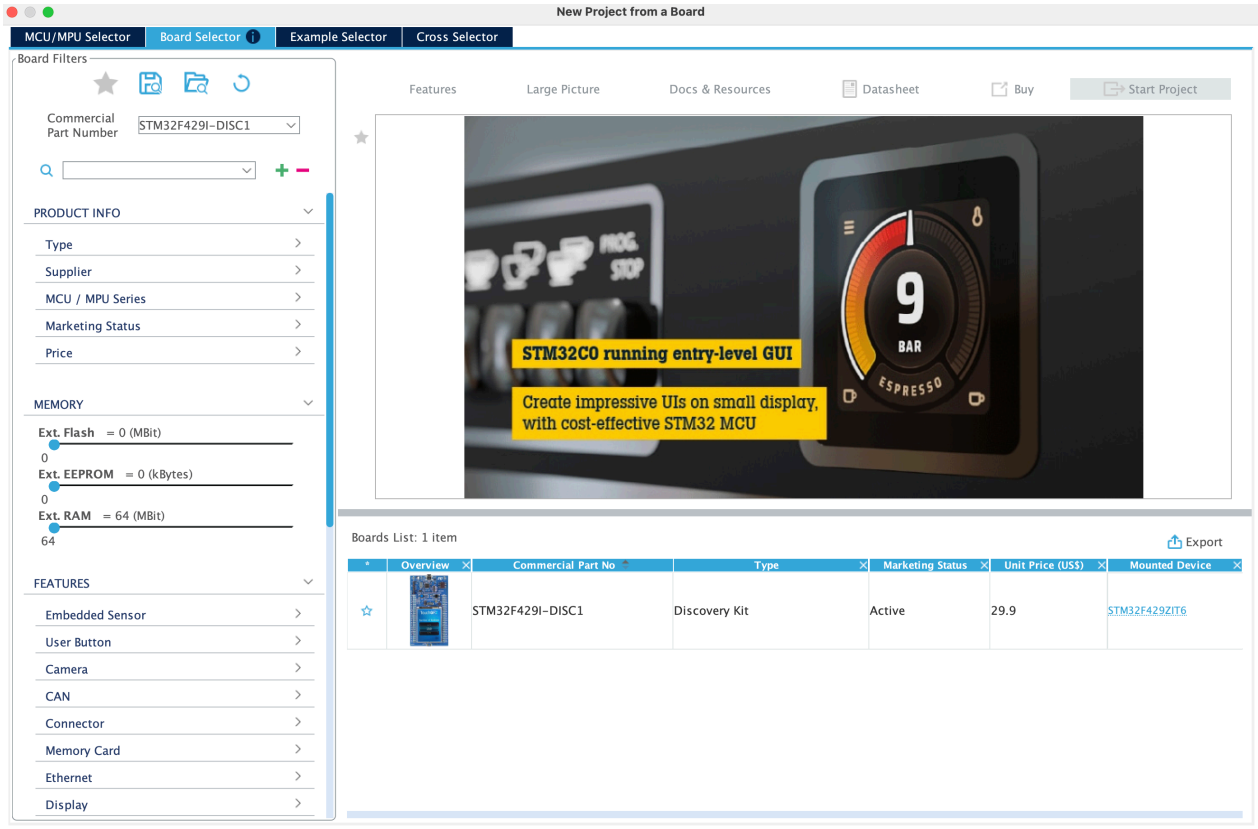


Fig. 2. Board selection window

The following steps in the board selection window should be made (Fig. 2):

1. Start to write “STM32F429I-DISC1” in the search edit box.
2. Select the necessary STM32F429I-DISC1 board from the list.
3. Press “*Start Project*” to be directed to the next configuration window (Fig. 4).

The pinout configuration window will be presented next (Fig. 3).

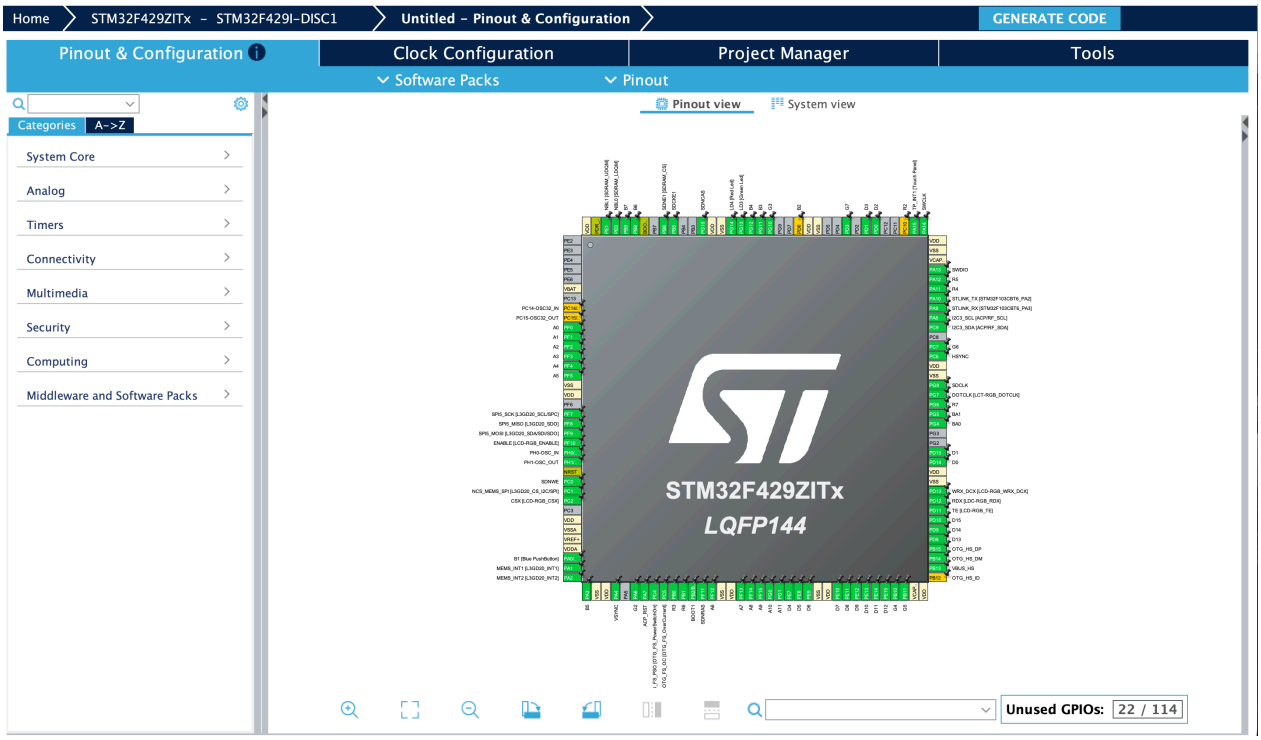


Fig. 3. Project configuration window

First, it is necessary to clear all the configuration of all pins of the microcontroller (Fig. 3): 1 – select *Pinout dropdown* menu; 2 – pick *Clear pinouts*. Then confirm the pinout clear operation (Fig. 4).

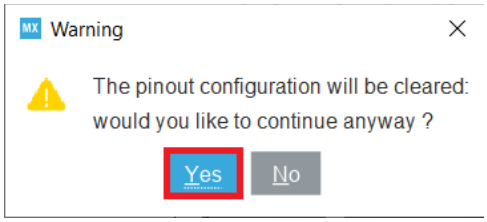


Fig. 4. Confirmation of pinouts clear

We will need two output pins and one input pin in the second laboratory work. The output pins will be used to control the LEDs, and the input pin will be used to register when the user button is pressed. The output pins are PG13—green LED and PIN_D14—red LED. The user button is connected to pin PA0. We will work at the register level in the second laboratory work and therefore will not perform any additional configuration of pins (Fig. 5).

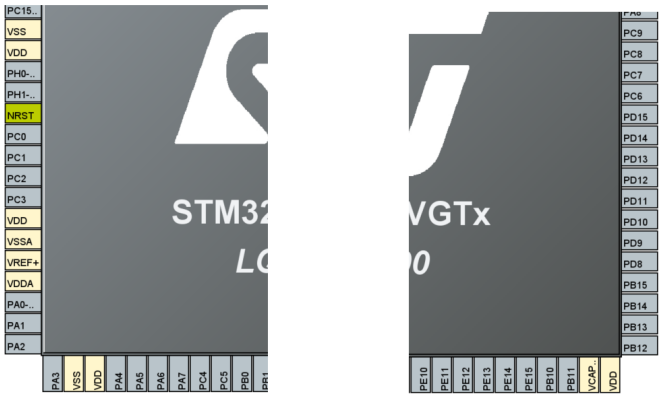


Fig. 5. No configuration of pins in graphical interface

Next step is to go to “Clock configuration” tab and to check if there are no errors (Fig. 6). There are usually no problems with the clock source, especially when performing basic operations with standard input output.

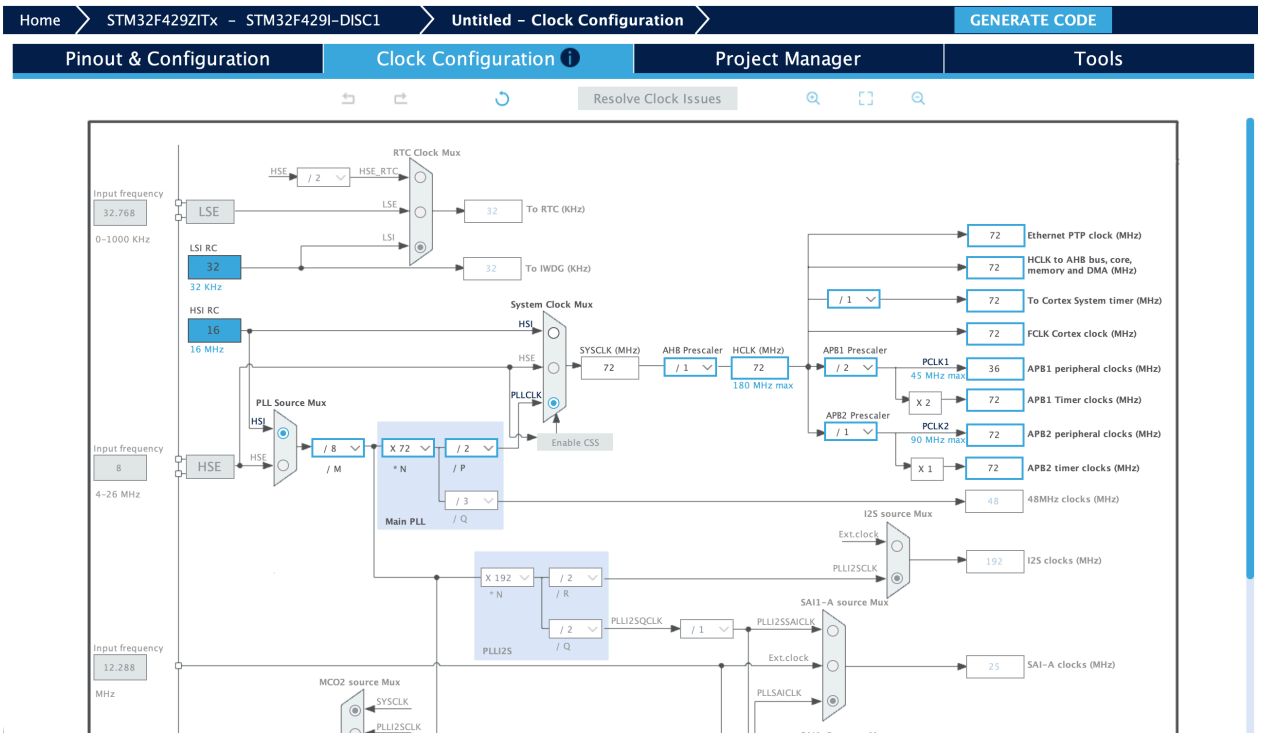


Fig. 6. Clock source configuration window

If there were any problems with clock source the red exclamation points would be shown in the Clock configuration window and the STM32CubeIDE will show the dialog window for automatic configuration of clock source (Figure 7).

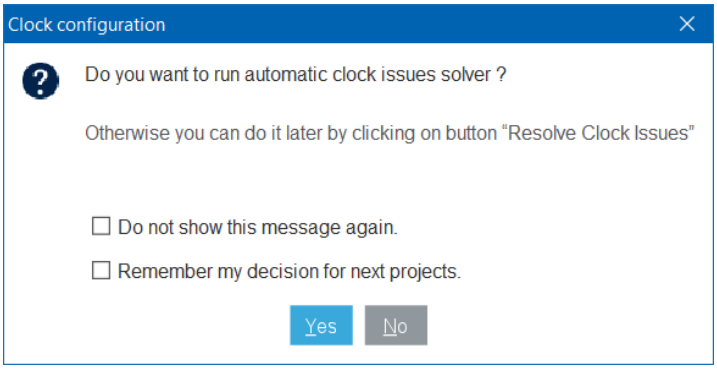


Fig. 7. Dialog window for selection of the automatic clock issues solver.

Automatic clock source matching can also be selected for the first laboratory work. We will talk about the selection of the clock source in detail in later lectures.

Next we will jump to the project configuration window (Fig. 8).

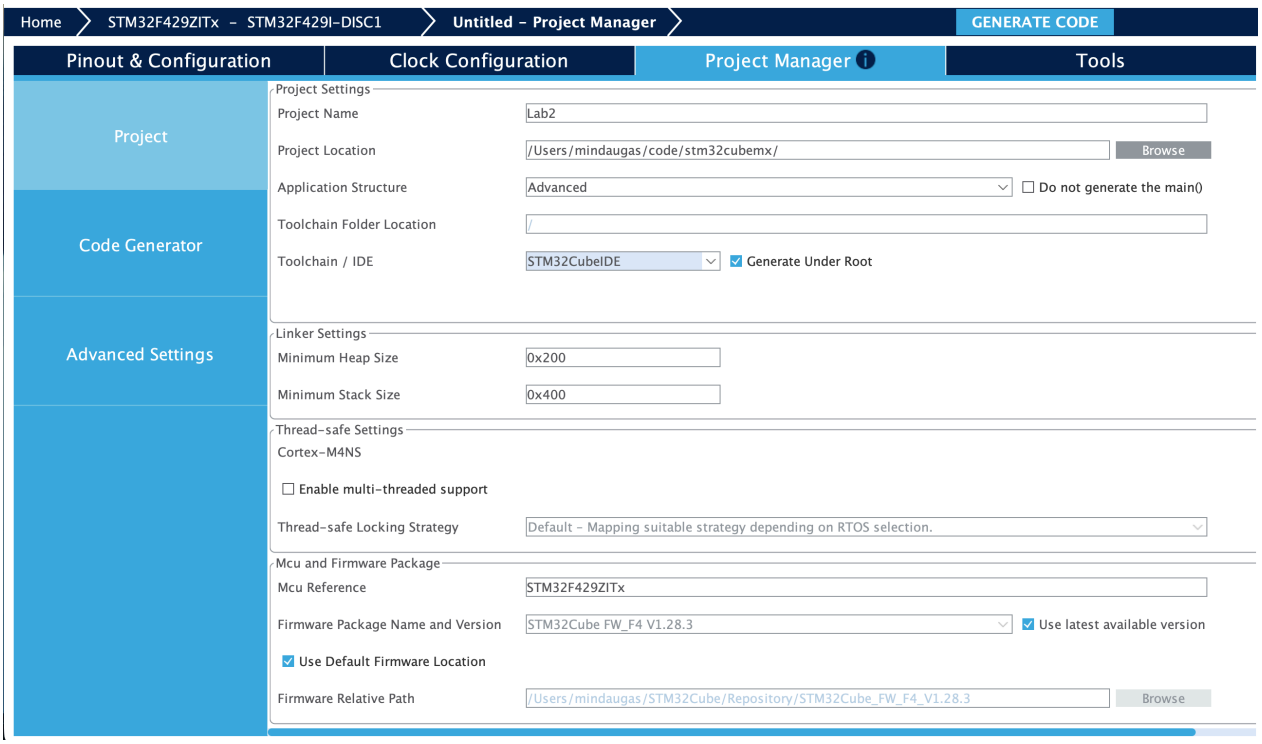


Fig. 8. Project configuration window

Update project settings with your project and location. Next, we click “Generate Code” button in the top right corner (Fig. 9).

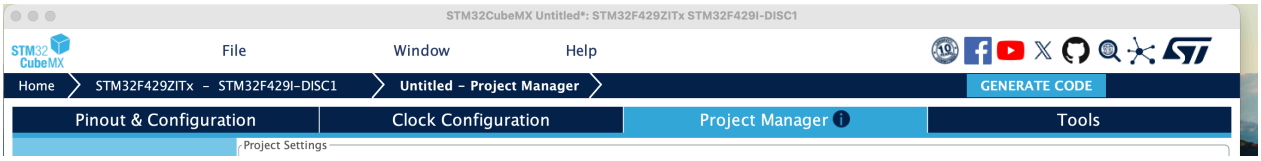


Fig. 9. Generate code

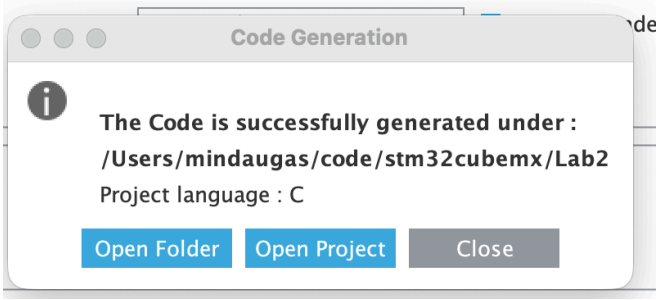


Fig. 10. Code generated

3.1.2. STM32CubeIDE

STM32CubeIDE is a software package to write code. First, when you run the STM32CubeIDE software for the first time, it will be necessary to specify the workspace directory (Fig. 11).

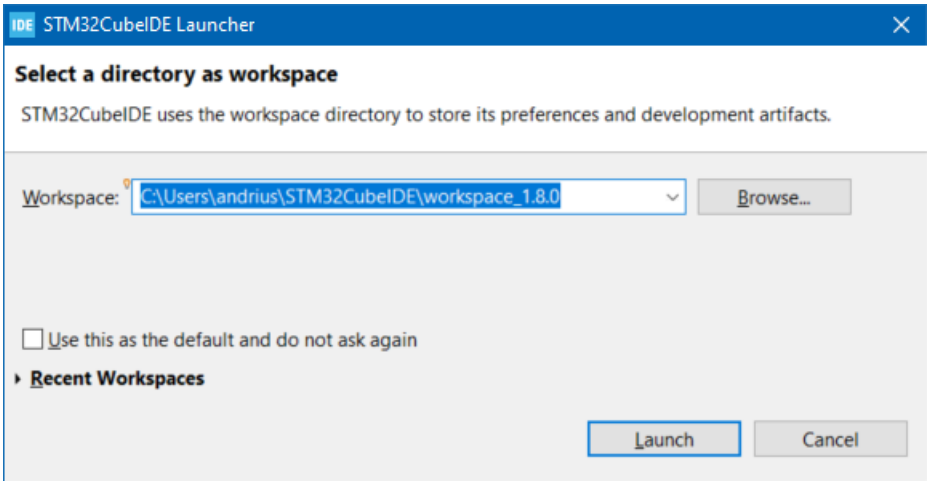


Fig. 11. The dialog window for selection of the workspace directory

For the programmer, the most important file to edit is the main.c file. Let's open the main.c file and start programming (Fig. 12).

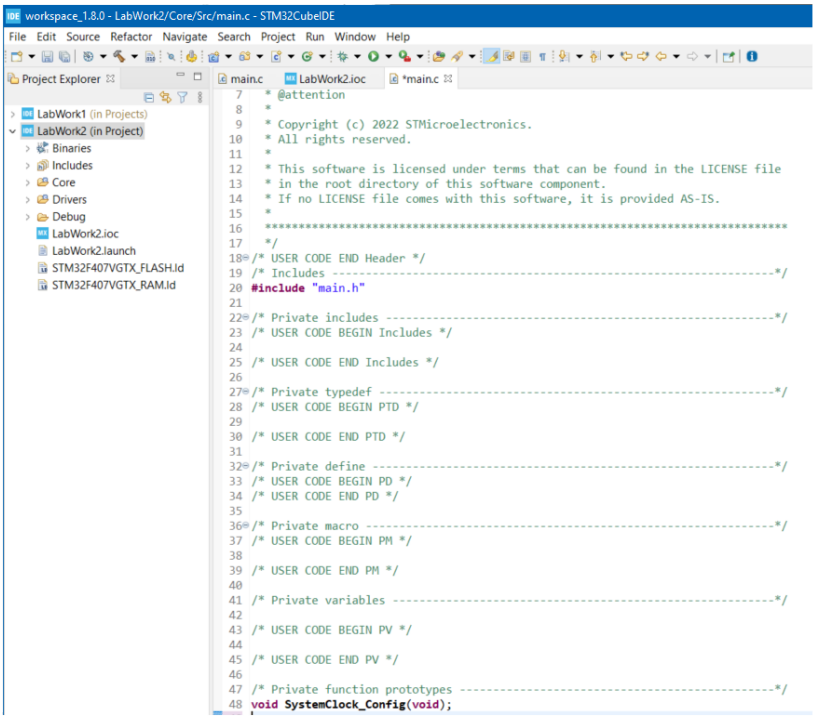


Fig. 12. The structure of the main.c file

Since we did not configure the pins in the graphical interface, there will be no *static void MX_GPIO_Init (void)* function in the generated project. It will be necessary to create this function by yourself. You will not be able to use the HAL Library functions in the second laboratory work. The second laboratory work will require working at the registers level and changing the content of the registers directly. However, in order not to make the laboratory work too complicated, you can use HAL_Delay (1000) HAL library function. You do not need to convert it to assembler.

3.1.3. Template of the code

The most important parts of the code are in blue color. It is necessary to understand that only parts of the code are presented. The blue code should be added in the correct places of *main.c* file.

```
...
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
void GPIO_Init(void);

...
```

```
/* USER CODE BEGIN 2 */
  GPIO_Init();
  /* USER CODE END 2 */

...

int main(void){
while (1){
    /* USER CODE BEGIN 3 */
        // Write your code
      }
  /* USER CODE END 3 */
}

...

/* USER CODE BEGIN 4 */
void GPIO_Init(void)
{
     // Write your code
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void){

...
```

3.1.4. Main functions and registers

```
// Delay function. In this particular case the delay of 100 ms
HAL_Delay(100);
```

RCC->AHB1ENR – peripheral clock enable register
GPIOD->MODER – port mode register
GPIOD->OTYPER – port output type register (push-pull, open-drain)
GPIOD->OSPEEDR – port output speed register
GPIOD->PUPDR – pull-up/pull-down register
GPIOD->ODR – port output data register
GPIOA->IDR – port input data register

Note. All tasks must be performed without using the HAL library functions:

- HAL_GPIO_Init()
- HAL_GPIO_ReadPin()
- HAL_GPIO_WritePin()
- HAL_GPIO_TogglePin()

In other words, the content of all necessary registers should be set directly.

The ***DM00031020 stm32f4x reference manual registers level*** literature source provided in the moodle system should be very valuable in this laboratory task.

3.3. Work by individual task

Students independently perform individual work assignments provided by the lecturer. Students can use any additional literature, but cannot talk to each other. When faced with difficulties and finding no solution, students can always ask specific questions to the teacher.

3.4. Literature

1. Slides of the lectures in the moodle.vgtu.lt system.
2. Code examples in the internet.
3. STM32F407. In the moodle system.
4. STM32F407 Discovery. In the moodle system.
5. UM1467 Getting started with software and firmware environments for the STM32F4DISCOVERY Kit. In the moodle system.
6. UM1472 Discovery kit for STM32F4. In the moodle system.
7. DM00031020 stm32f4x reference manual registers level. In the moodle system.